



PyQuest

Information- & Aufgabenblatt

Kapitel 4: String-Funktionen

Text durchsuchen, zerlegen, verbinden und umformen – die wichtigsten Werkzeuge für Strings.

Länge und Schreibweise

Ein **String** heißt auf Deutsch **Zeichenkette** – eine Kette aus einzelnen Zeichen. Strings gehören zu den am häufigsten benutzten Datentypen – Python bringt viele eingebaute **Funktionen und Methoden** mit, um Text zu untersuchen und zu verändern. Los geht's mit den einfachsten: Länge messen und Groß-/Kleinschreibung ändern.

Mit **len(text)** erfährst du, aus wie vielen Zeichen ein String besteht (Leerzeichen zählen mit!).

len() zählt alle Zeichen, auch das Leerzeichen:

Beispiel

```
text = "Hello World"
print(len(text))
```

Mit den **Methoden** .upper() und .lower() (Punktschreibweise, wie du sie schon von .append() bei Listen kennst) wandelst du einen String komplett in Groß- bzw. Kleinbuchstaben um. Der **ursprüngliche** String bleibt dabei unverändert – die Methode gibt einen **neuen** String zurück.

upper() und lower() erzeugen jeweils einen neuen String:

Beispiel

```
text = "Hello World"
print(text.upper())
print(text.lower())
```

Aufgabe 1: Was gibt len("Python") zurück?

- ☐ a) 5
- ☐ b) 6
- ☐ c) 7



Aufgabe 2: In der Variable `satz` steht ein Text. Gib zuerst seine Länge aus (`len(...)`), dann den Text in Großbuchstaben (`.upper()`).

```
satz = "Ich liebe Python"
```

Dein Code:

Suchen in Strings

Mit dem **in**-Operator prüfst du, ob ein Teilstring in einem String **vorkommt**. Das Ergebnis ist ein `bool`: `True` oder `False`. Du kennst `in` schon aus Listen – bei Strings funktioniert es genauso.

`in` prüft, ob "World" in `text` enthalten ist:

Beispiel

```
text = "Hello World"
enthalten = "World" in text
print(enthalten)
```

Willst du wissen, **an welcher Position** ein Teilstring beginnt, nutzt du `.find(teilstring)`. Sie gibt den **Index** des ersten Treffers zurück – oder `-1`, wenn nichts gefunden wurde.

`find()` liefert den Startindex des Treffers:

Beispiel

```
text = "Hello World"
position = text.find("World")
print(position)
```

Aufgabe 3: Was gibt `"Hallo".find("xyz")` zurück, wenn "xyz" nicht im String vorkommt?

- ☐ a) 0
- ☐ b) -1
- ☐ c) None



Aufgabe 4: In `email` steht eine E-Mail-Adresse. Prüfe mit `in`, ob sie das Zeichen `@` enthält, und gib das Ergebnis aus. Gib danach mit `.find("@")` die Position des `@`-Zeichens aus.

```
email = "ada@python.de"
```

Dein Code:

Teilstrings mit Slicing

Mit **Slicing** (`string[start:ende]`) schneidest du einen **Teilstring** heraus. Genau wie bei `range()` ist der Index **start** dabei, der Index **ende** aber **nicht mehr** – er wird ausgeschlossen.

Jedes Zeichen im String hat einen Index, beginnend bei **0** – genau wie bei Listen.

`text[6:11]` holt die Zeichen an den Positionen 6 bis 10:

Beispiel

```
text = "Hello World"
teil = text[6:11]
print(teil)
```

Lässt du eine Seite **weg**, nimmt Python automatisch den Anfang bzw. das Ende:

- `text[:5]` → alles **vom Anfang bis** Index 4
- `text[6:]` → alles **ab** Index 6 **bis zum Ende**

Auch **negative Indizes** funktionieren: `text[-1]` ist das letzte Zeichen.

Aufgabe 5: Was gibt `"Wasserfallreiniger"[4:9]` zurück?

- ☐ a) erfal
- ☐ b) serfa
- ☐ c) wasse



Aufgabe 6: In seriennummer steht der Text "SE7133131855". Extrahiere mit Slicing: 1. Die ersten 2 Zeichen (das Druckerei-Kürzel) → gib sie aus 2. Alles ohne die ersten 2 und ohne das letzte Zeichen (die eigentliche Nummer) → gib sie aus

```
seriennummer = "SE7133131855"
```

Dein Code:

Ersetzen und Teilen

Mit `.replace(alt, neu)` ersetzt du in einem String alle Vorkommen von `alt` durch `neu`. Auch hier gilt: Der ursprüngliche String bleibt unverändert, du bekommst einen **neuen** String zurück.

`replace()` ersetzt jedes Vorkommen von "World":

Beispiel

```
text = "Hello World"
neu = text.replace("World", "Universe")
print(neu)
```

Mit `.split(trennzeichen)` zerlegst du einen String an jedem Vorkommen des Trennzeichens in eine **Liste** von Teilstrings. Das ist super praktisch, um Text mit fester Struktur (z. B. "Nachname | Vorname | Ort") zu zerlegen.

`split(",")` zerlegt den String an jedem Komma:

Beispiel

```
text = "Hello, World"
teile = text.split(",")
print(teile)
```

**Aufgabe 7: Was für einen Datentyp gibt `.split(...)` zurück?**

- ☐ a) Einen String
- ☐ b) Eine Liste
- ☐ c) Ein Dictionary

Aufgabe 8: In datensatz steht "Klein|Max|Berlin". Zerlege den Text mit `.split("|")` und speichere das Ergebnis in teile. Gib teile aus.

Erwartet: ['Klein', 'Max', 'Berlin']

```
datensatz = "Klein|Max|Berlin"
```

Dein Code:

Verbinden mit `join()`

`.join()` ist quasi das **Gegenteil** von `.split()`: Es fügt die Elemente einer **Liste** zu einem **einzigem String** zusammen, getrennt durch ein Trennzeichen.

Achtung, ungewohnte Reihenfolge: Das Trennzeichen steht **vor** dem Punkt, die Liste kommt als Argument **in die Klammer**: `"trennzeichen".join(liste)`.

Die Wörter werden mit einem Leerzeichen verbunden:

Beispiel

```
woerter = ["Hello", "World"]  
text = " ".join(woerter)  
print(text)
```



Du kannst als Trennzeichen **jeden** String verwenden – auch mehrere Zeichen oder gar keins (""). So lässt sich `split()` und `join()` auch **kombinieren**, um Text umzuformatieren: erst zerlegen, dann in neuer Form wieder zusammensetzen.

Aufgabe 9: Was gibt `"` und `".join(["Katze", "Hund", "Maus"])` zurück?

- ☐ a) Katze und Hund und Maus
- ☐ b) Katze, Hund, Maus
- ☐ c) ['Katze', 'Hund', 'Maus']

Aufgabe 10: In namen steht die Liste `["Ada", "Alan", "Grace"]`. Verbinde die Namen mit , (Komma + Leerzeichen) zu einem einzigen String und gib ihn aus.

Erwartet: Ada, Alan, Grace

```
namen = ["Ada", "Alan", "Grace"]
```

Dein Code:

String-Funktionen anwenden

Jedes Zeichen hat intern einen **Zahlencode** (den ASCII-/Unicode-Wert). Mit `ord(zeichen)` bekommst du diesen Code, mit `chr(zahl)` umgekehrt das Zeichen zu einem Code.

Damit lässt sich zum Beispiel die **Position eines Buchstabens im Alphabet** berechnen: `ord('A')` ist 65, `ord('B')` ist 66, usw. Ziehst du `ord('A')` ab und addierst 1, bekommst du die Position (A = 1, B = 2, ...).

Position von 'C' im Alphabet berechnen:

Beispiel

```
buchstabe = "C"
position = ord(buchstabe.upper()) - ord("A") + 1
print(position)
```



Aufgabe 11: Frage mit Buchstabe: einen einzelnen Buchstaben ab. Berechne seine Position im Alphabet (A = 1, B = 2, ...) und gib nur die Zahl aus. Achte darauf, die Eingabe zuerst mit `.upper()` in einen Großbuchstaben umzuwandeln.

Beispiel: Eingabe b → 2

Dein Code:

Jetzt kombinierst du `split()` und `join()`: Ein Datensatz im Format Nachname|Vorname|Personalnummer|Wohnort soll umsortiert werden, sodass die Personalnummer **vorne** steht: Personalnummer|Nachname|Vorname|Wohnort.

Erst zerlegst du den String mit `split()` in seine Teile, dann baust du ihn in der **neuen Reihenfolge** mit `join()` wieder zusammen.

Aufgabe 12: In datensatz steht "Klein|Max|XZ_1234|Berlin" im Format Nachname|Vorname|Personalnummer|Wohnort.

1. Zerlege den String mit `.split("|")` in eine Liste teile.
2. Baue daraus eine **neue Liste** in der Reihenfolge [Personalnummer, Nachname, Vorname, Wohnort].
3. Verbinde sie mit `"|".join(...)` zu einem String und gib ihn aus.

Erwartet: **XZ_1234|Klein|Max|Berlin**

`datensatz = "Klein|Max|XZ_1234|Berlin"`

Dein Code: